

Proyectos completos

Cómo analizar una solución AI aplicada

Cada caso integrador puede entenderse como la combinación de cinco capas:

Capa	Pregunta guía
Interfaz	¿Cómo interactúa la persona con la solución?
Entrada de datos	¿Qué información ingresa al sistema?
Workflow n8n	¿Qué acciones se automatizan?
Base de datos / apps	¿Dónde se guarda o consulta información?
Salida	¿Qué respuesta, acción o resultado recibe el usuario o equipo?

Proyecto 1: Sistema de Captura de *Leads* con Seguimiento

Caso de negocio: automatización del proceso comercial inicial.

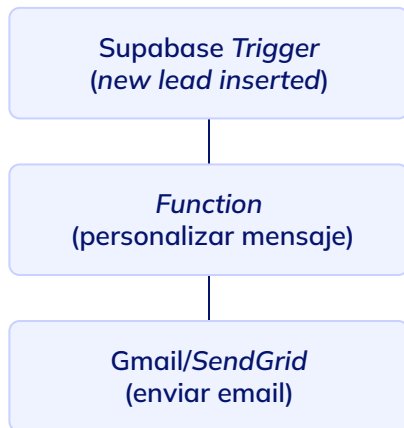
Este caso integra una interfaz de captura, un *workflow* de procesamiento y una base de datos para seguimiento comercial.

El usuario completa un formulario. n8n recibe los datos, los valida, los guarda en Supabase, envía un *email* de bienvenida y notifica al equipo comercial. A partir de ese registro, el *lead* puede ser actualizado, consultado o usado en futuras automatizaciones.

Te recomendamos ver la implementación completa en el manual de Desarrollo UI (Formulario de *Onboarding*).

Características adicionales para completar el proyecto:

Email de Bienvenida Automático (n8n workflow):



Notificación a Slack:

Slack Node

- *Channel:* #leads
- *Message:* "Nuevo lead:
{{ \$json.first_name }}
{{ \$json.last_name }} de
{{ \$json.company_name }}"



Proyecto 2: *Dashboard* de Métricas en Tiempo Real

Descripción

Caso de negocio: seguimiento operativo o comercial

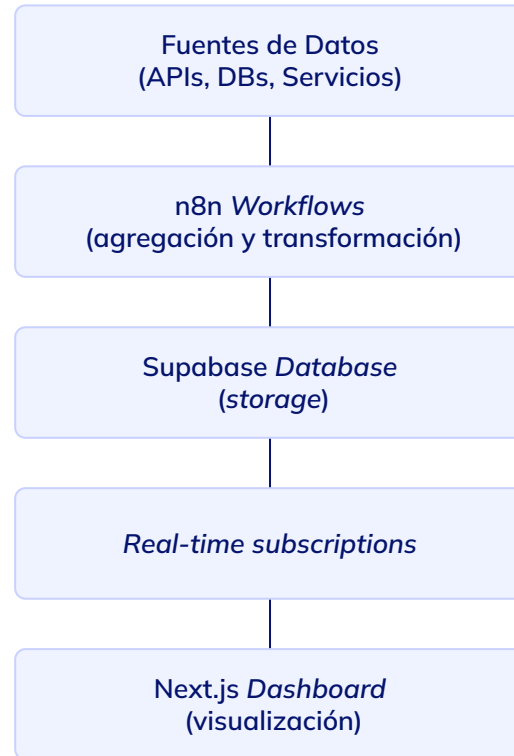
Un *dashboard* de métricas permite que un equipo interno consulte información actualizada sin revisar manualmente distintas herramientas.

En esta solución, n8n recolecta datos desde APIs, bases o servicios externos, los transforma y los guarda en Supabase. Luego, una interfaz visual muestra los indicadores principales para facilitar el seguimiento y la toma de decisiones.

Stack Tecnológico

- *Frontend*: Next.js + shadcn/ui + *Recharts*
- *Backend*: n8n *workflows*
- *Database*: Supabase
- *Deployment*: Vercel

Arquitectura



Implementación Paso a Paso

Paso 1: Esquema de Base de Datos

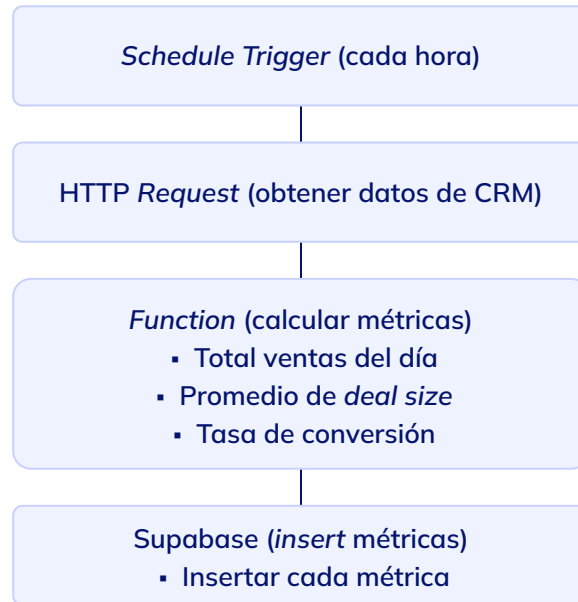
```
-- Tabla de métricas
CREATE TABLE metrics (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  metric_name TEXT NOT NULL,
  metric_value NUMERIC NOT NULL,
  metric_type TEXT, -- 'count', 'currency', 'percentage', 'duration'
  category TEXT, -- 'sales', 'marketing', 'operations', 'support'
  metadata JSONB,
  recorded_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

...

```
CREATE INDEX idx_metrics_name ON metrics(metric_name);
CREATE INDEX idx_metrics_category ON metrics(category);
CREATE INDEX idx_metrics_recorded_at ON metrics(recorded_at DESC);
-- Vista para últimas métricas por nombre
CREATE VIEW latest_metrics AS
SELECT DISTINCT ON (metric_name)
    *
FROM metrics
ORDER BY metric_name, recorded_at DESC;
-- Tabla de alerts
CREATE TABLE metric_alerts (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    metric_name TEXT NOT NULL,
    condition TEXT, -- 'above', 'below', 'change'
    threshold NUMERIC,
    message TEXT,
    severity TEXT DEFAULT 'info', -- 'info', 'warning', 'error'
    is_active BOOLEAN DEFAULT true
);
```


Paso 2: *Workflows* de n8n para Recolección de Datos

Workflow A: Métricas de Ventas (corre cada hora)



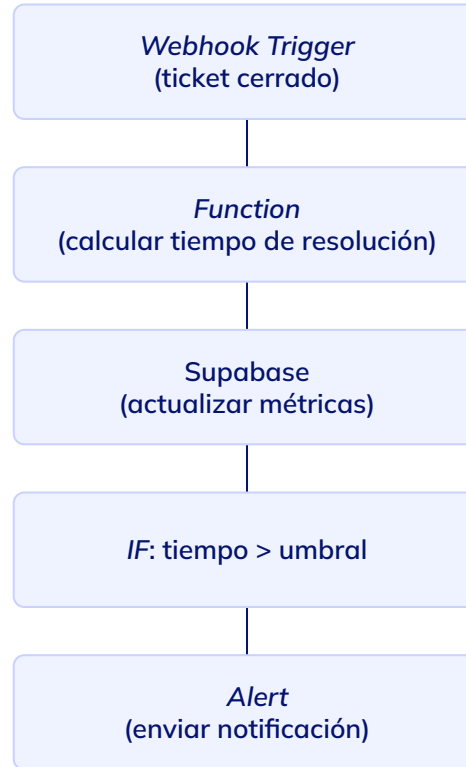
Function Node - Calcular Métricas:

```
const deals = $json.deals;
const metrics = [];
// Total ventas del día
const todayDeals = deals.filter(d => {
  const dealDate = new Date(d.created_at);
  const today = new Date();
  return dealDate.toDateString() === today.toDateString();
});
const totalSales = todayDeals.reduce((sum, d) => sum + d.amount, 0);
metrics.push({
  metric_name: 'total_sales_today',
  metric_value: totalSales,
  metric_type: 'currency',
  category: 'sales',
  metadata: { count: todayDeals.length }
});
// Promedio deal size
const avgDealSize = totalSales / todayDeals.length || 0;
```

...

```
metrics.push({
  metric_name: 'avg_deal_size',
  metric_value: avgDealSize,
  metric_type: 'currency',
  category: 'sales'
});
// Tasa de conversión
const wonDeals = todayDeals.filter(d => d.status === 'won').length;
const conversionRate = (wonDeals / todayDeals.length * 100) || 0;
metrics.push({
  metric_name: 'conversion_rate',
  metric_value: conversionRate,
  metric_type: 'percentage',
  category: 'sales'
});
return metrics.map(m => ({ json: m }));
```

Workflow B: Métricas de Soporte (real-time)



Paso 3: Componente de *Dashboard*

```
// app/dashboard/page.tsx
'use client'
import { useEffect, useState } from 'react'
import { supabase } from '@lib/supabase'
import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card'
import { ArrowUpIcon, ArrowDownIcon, TrendingUpIcon } from 'lucide-react'
import { Line, LineChart, ResponsiveContainer, Tooltip, XAxis, YAxis } from 'recharts'

interface Metric {
  id: string
  metric_name: string
  metric_value: number
  metric_type: string
  category: string
  recorded_at: string
}
```

...

```
export default function MetricsDashboard() {
  const [metrics, setMetrics] = useState<Record<string, Metric>>({})
  const [history, setHistory] = useState<Metric[]>([])
  const [loading, setloading] = useState(true)

  useEffect(() => {
    fetchMetrics()
    fetchHistory()
    subscribeToMetrics()
  }, [])

  const fetchMetrics = async () => {
    const { data } = await supabase.from('latest_metrics').select('*')
    if (data) {
      const metricsMap = data.reduce((acc, metric) => {
        acc[metric.metric_name] = metric
        return acc
      }, {} as Record<string, Metric>)
      setMetrics(metricsMap)
    }
  }
}
```

...

...

```
    setLoading(false)
  }
}

const fetchHistory = async () => {
  const oneDayAgo = new Date()
  oneDayAgo.setDate(oneDayAgo.getDate() - 1)
  const { data } = await supabase
    .from('metrics')
    .select('*')
    .gte('recorded_at', oneDayAgo.toISOString())
    .order('recorded_at', { ascending: true })
  if (data) setHistory(data)
}

const subscribeToMetrics = () => {
  const subscription = supabase
    .channel('metrics-realtime')
    .on(
      'postgres_changes',
```

...

...

```
{ event: 'INSERT', schema: 'public', table: 'metrics' },
(payload) => {
  const newMetric = payload.new as Metric
  setMetrics((current) => ({ ...current, [newMetric.metric_name]: newMetric }))
  setHistory((current) => [...current, newMetric])
  showNotification(newMetric)
}
)
.subscribe()
return () => subscription.unsubscribe()
}

const showNotification = (metric: Metric) => {
  if ('Notification' in window && Notification.permission === 'granted') {
    new Notification('Métrica Actualizada', {
      body: `${metric.metric_name}: ${formatValue(metric)}`,
      icon: '/icon.png'
    })
  }
}
```

...

...

```
}  
const formatValue = (metric: Metric) => {  
  switch (metric.metric_type) {  
    case 'currency':  
      return new Intl.NumberFormat('es-ES', { style: 'currency', currency: 'USD'  
}).format(metric.metric_value)  
    case 'percentage':  
      return `${metric.metric_value.toFixed(2)}%`  
    case 'duration':  
      return `${metric.metric_value.toFixed(0)} min`  
    default:  
      return metric.metric_value.toString()  
  }  
}  
  
const getChartData = (metricName: string) => {  
  return history  
    .filter(m => m.metric_name === metricName)  
    .map(m => ({
```

...

...

```
    time: new Date(m.recorded_at).toLocaleTimeString('es-ES', { hour: '2-digit', minute: '2-digit' })),  
    value: m.metric_value  
  )))  
}  
  
if (loading) {  
  return <div className="flex items-center justify-center h-screen">Cargando...</div>  
}  
  
return (  
  <div className="p-8 space-y-8">  
    <div className="flex items-center justify-between">  
      <h1 className="text-3xl font-bold">Dashboard de Métricas</h1>  
      <div className="text-sm text-muted-foreground">  
        Actualización en tiempo real  
        <span className="inline-block w-2 h-2 ml-2 bg-green-500 rounded-full animate-pulse" />  
      </div>  
    </div>  
  </div>  
)
```

...

...

```
</div>
<div className="grid gap-4 md:grid-cols-2 lg:grid-cols-4">
  <MetricCard title="Ventas Hoy" metric={metrics.total_sales_today} formatValue={formatValue} />
  <MetricCard title="Tasa de Conversión" metric={metrics.conversion_rate} formatValue={formatValue} />
  <MetricCard title="Leads Nuevos" metric={metrics.new_leads_today} formatValue={formatValue} />
  <MetricCard title="Tickets Abiertos" metric={metrics.open_tickets} formatValue={formatValue} />
</div>
<div className="grid gap-4 md:grid-cols-2">
  <Card>
    <CardHeader><CardTitle>Ventas - Últimas 24h</CardTitle></CardHeader>
    <CardContent>
      <ResponsiveContainer width="100%" height={300}>
        <LineChart data={getChartData('total_sales_today')}>
          <XAxis dataKey="time" />
          <YAxis />
        </LineChart>
      </ResponsiveContainer>
    </CardContent>
  </Card>
</div>
```

...

...

```
        <Tooltip />
        <Line type="monotone" dataKey="value" stroke="#3b82f6" strokeWidth={2} />
      </LineChart>
    </ResponsiveContainer>
  </CardContent>
</Card>
<Card>
  <CardHeader><CardTitle>Tasa de Conversión - Últimas 24h</CardTitle></CardHeader>
  <CardContent>
    <ResponsiveContainer width="100%" height={300}>
      <LineChart data={getChartData('conversion_rate')}>
        <XAxis dataKey="time" />
        <YAxis />
        <Tooltip />
        <Line type="monotone" dataKey="value" stroke="#10b981" strokeWidth={2} />
      </LineChart>
    </ResponsiveContainer>
```

...

...

```
        </CardContent>
      </Card>
    </div>
  </div>
)
```

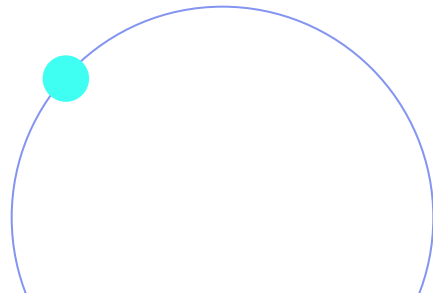
```
}

function MetricCard({ title, metric, formatValue }: any) {
  if (!metric) return null
  return (
    <Card>
      <CardHeader className="flex flex-row items-center justify-between pb-2">
        <CardTitle className="text-sm font-medium">{title}</CardTitle>
        <TrendingUpIcon className="w-4 h-4 text-muted-foreground" />
      </CardHeader>
      <CardContent>
        <div className="text-2xl font-bold">{formatValue(metric)}</div>
        <p className="text-xs text-muted-foreground mt-1">
          Actualizado: {new
```

...

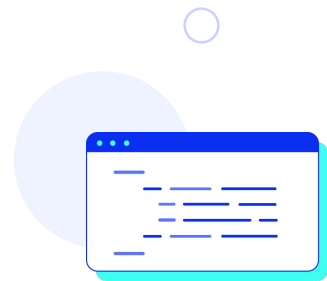
...

```
Date(metric.recorded_at).toLocaleTimeString('es-ES'))  
    </p>  
  </CardContent>  
</Card>  
)  
}
```



Paso 4: *Deployment*

```
# Build y deploy
npm run build
vercel --prod
# Configurar variables de entorno en Vercel
# NEXT_PUBLIC_SUPABASE_URL
# NEXT_PUBLIC_SUPABASE_ANON_KEY
```



Proyecto 3: Chat Interface con IA

Descripción

Interfaz de chat que conecta con *workflows* de n8n para procesamiento con LLMs, almacenando conversaciones en Supabase.

Esquema de Base de Datos

```
CREATE TABLE conversations (  
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  user_id UUID,  
  title TEXT,  
  status TEXT DEFAULT 'active'  
);
```


...

```
CREATE TABLE messages (  
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  conversation_id UUID REFERENCES conversations(id) ON DELETE CASCADE,  
  role TEXT NOT NULL, -- 'user', 'assistant', 'system'  
  content TEXT NOT NULL,  
  metadata JSONB  
);  
  
CREATE INDEX idx_messages_conversation ON messages(conversation_id, created_at);
```

Componente de Chat

```
// components/ChatInterface.tsx
'use client'
import { useState, useEffect, useRef } from 'react'
import { supabase } from '@lib/supabase'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { Card } from '@components/ui/card'
import { SendIcon, Loader2 } from 'lucide-react'

interface Message {
  id: string
  role: 'user' | 'assistant'
  content: string
  created_at: string
}

export default function ChatInterface() {
  const [messages, setMessages] = useState<Message[]>([])
```

...

```
const [input, setInput] = useState('')
const [loading, setLoading] = useState(false)
const [conversationId, setConversationId] = useState<string | null>(null)
const messagesEndRef = useRef<HTMLDivElement>(null)

useEffect(() => {
  initConversation()
}, [])

useEffect(() => {
  scrollToBottom()
}, [messages])

const initConversation = async () => {
  const { data } = await supabase.from('conversations').insert([{ title: 'Nueva conversación'
}]).select().single()
  if (data) setConversationId(data.id)
}

const sendMessage = async () => {
```

...

...

```
if (!input.trim() || !conversationId) return
const userMessage = { conversation_id: conversationId, role: 'user', content: input }
const { data: savedMessage } = await supabase.from('messages').insert([userMessage]).select().single()
setMessages(prev => [...prev, savedMessage])
setInput('')
setLoading(true)
try {
  const response = await fetch('/api/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ conversation_id: conversationId, message: input, history: messages })
  })
  const { reply } = await response.json()
  const assistantMessage = { conversation_id: conversationId, role: 'assistant', content: reply }
  const { data: assistantMsg } = await
supabase.from('messages').insert([assistantMessage]).select().single()
```

...

...

```
    setMessages(prev => [...prev, assistantMsg])
  } catch (error) {
    console.error('Error:', error)
  } finally {
    setLoading(false)
  }
}

const scrollToBottom = () => {
  messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' })
}

return (
  <Card className="flex flex-col h-[600px] max-w-2xl mx-auto">
    <div className="flex-1 overflow-y-auto p-4 space-y-4">
      {messages.map((message) => (
        <div key={message.id} className={`flex ${message.role === 'user' ? 'justify-end' :
'justify-start'}`}>
          <div className={`max-w-[80%] rounded-lg p-3 ${message.role === 'user' ? 'bg-blue-600 text-white' :
'bg-gray-100 text-gray-900'}`}>>

```

...

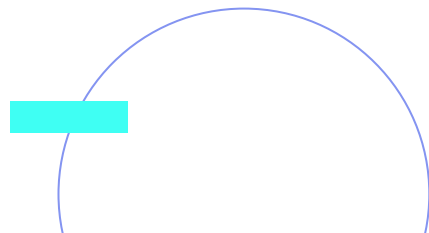
...

```
{message.content}
</div>
</div>
)}}
{loading && (
  <div className="flex justify-start">
    <div className="bg-gray-100 rounded-lg p-3">
      <Loader2 className="w-4 h-4 animate-spin" />
    </div>
  </div>
)}
<div ref={messagesEndRef} />
</div>
<div className="border-t p-4">
  <div className="flex gap-2">
    <Input
      value={input}
      onChange={(e) => setInput(e.target.value)}
      onPress={(e) => e.key === 'Enter' && sendMessage()}
      placeholder="Escribe tu mensaje..."
```

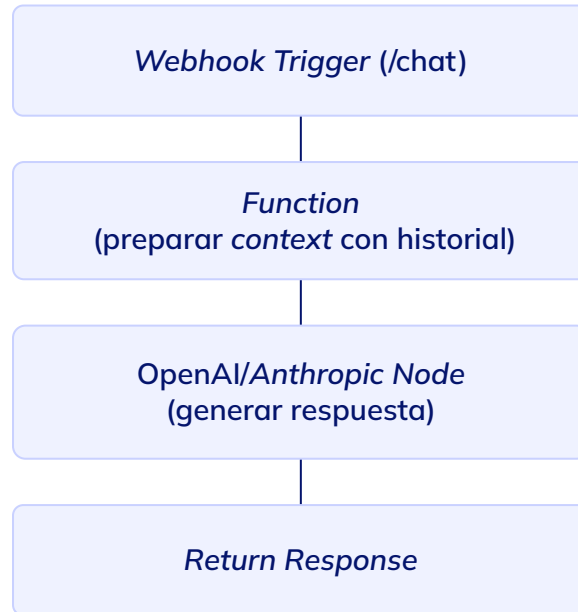
...

...

```
        disabled={loading}  
      />  
      <Button onClick={sendMessage} disabled={loading}>  
        <SendIcon className="w-4 h-4" />  
      </Button>  
    </div>  
  </div>  
</Card>  
)  
}
```



n8n Workflow para Chat



Mejores prácticas para soluciones AI aplicadas

Diseño del *workflow*

El *workflow* debe estar diseñado para resolver un objetivo de negocio concreto. Antes de construirlo, conviene definir **qué dispara el proceso, qué datos ingresan, qué herramientas intervienen y qué resultado se espera**.

Buenas prácticas

- Definir claramente el trigger del *workflow*.
- Separar el flujo en etapas comprensibles.
- Validar los datos antes de ejecutar acciones importantes.
- Evitar nodos innecesarios o duplicados.
- Usar nombres claros para nodos y variables.
- Documentar qué hace cada parte del flujo.
- Probar el *workflow* con datos ficticios antes de usar datos reales.

Datos y calidad de la información

Las soluciones automatizadas dependen de la calidad de los datos que reciben. Si la información llega incompleta, duplicada o mal estructurada, el *workflow* puede ejecutar acciones incorrectas.

Buenas prácticas

- Definir campos obligatorios y opcionales.
- Normalizar datos antes de guardarlos.
- Validar formatos, como *emails*, fechas o números.

- Evitar duplicados mediante identificadores claros.
- Registrar el origen de los datos.
- Guardar solo la información necesaria.
- Revisar qué datos deben persistir y cuáles son temporales.

Por ejemplo, en un flujo de captura de *leads*, n8n puede limpiar el *email*, completar un estado inicial, registrar la fuente del formulario y evitar crear un *lead* duplicado si el correo ya existe.

Seguridad y credenciales

Cuando una solución conecta interfaces, bases de datos, APIs y modelos de IA, también aumenta la responsabilidad sobre la seguridad.

Las credenciales, *webhooks* y datos sensibles deben protegerse desde el diseño.

Buenas prácticas

- No exponer credenciales en el código.
- Usar variables de entorno.
- Proteger URLs de *webhooks* cuando corresponda.
- Configurar permisos mínimos necesarios.
- Revisar políticas de acceso en bases de datos.
- Evitar guardar datos sensibles si no son necesarios.
- Separar entornos de prueba y producción.
- Controlar quién puede modificar *workflows* productivos.

Manejo de errores y respuestas

Toda solución automatizada debe contemplar que algo puede fallar: una API puede no responder, un dato puede llegar incompleto, una credencial puede vencer o un modelo de IA puede devolver una respuesta insuficiente.

Buenas prácticas

- Incluir mensajes de error claros.
- Registrar fallos en *logs* o bases de datos.
- Definir rutas alternativas cuando una integración falla.

- Notificar al equipo si ocurre un error crítico.
- Evitar que el usuario quede sin respuesta.
- Probar casos incompletos o incorrectos.
- Devolver respuestas comprensibles desde n8n hacia la interfaz.

Por ejemplo, si falla la conexión con el CRM, el *workflow* puede guardar el *lead* en Supabase, notificar al equipo comercial y marcar el registro como “pendiente de sincronización”.

Uso responsable de IA y LLMs

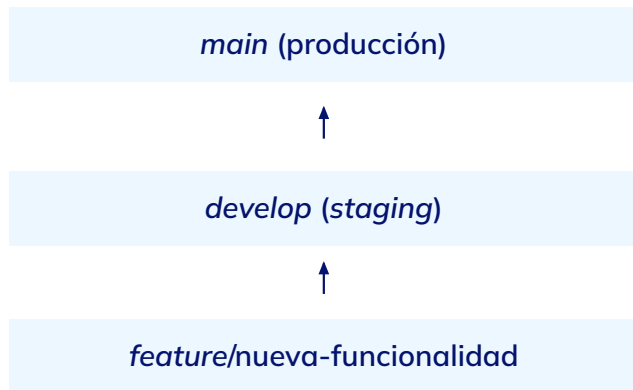
Cuando el *workflow* incorpora modelos como OpenAI, Claude o Gemini, es importante recordar que **la IA no debe operar sin criterios de control**. Los modelos pueden generar respuestas útiles, pero también pueden equivocarse, inventar información o responder de forma inconsistente.

Buenas prácticas

- Definir *prompts* claros y específicos.
- Indicar el contexto y el formato esperado de respuesta.
- Validar respuestas antes de ejecutar acciones críticas.
- Evitar que el modelo tome decisiones sensibles sin revisión humana.
- Registrar interacciones importantes.
- Considerar límites de costo, latencia y uso.
- Usar datos confiables como contexto.
- Incorporar aprobación humana cuando el riesgo sea alto.

Git Workflows

Estrategia de *Branching*:



Convenciones de *Commits*:

<i>feat</i>	nueva funcionalidad
<i>fix</i>	corrección de <i>bug</i>
<i>docs</i>	documentación
<i>style</i>	formateo
<i>refactor</i>	refactorización
<i>test</i>	tests
<i>chore</i>	tareas de mantenimiento

Ejemplo:

```
git commit -m "feat: add real-time metrics dashboard"  
git commit -m "fix: resolve webhook timeout issue"  
git commit -m "docs: update API integration guide"
```

